

DFT 面霸真题

1: Boundary Scan 是什么？应用场景是什么？实现的方法是什么？挑战是什么？

答: Boundary Scan 就是边界扫描, 是由 Joint Test action Group 起草的规范, 最初是为了解决板级芯片之间的互联测试的问题, 实现方法就是在芯片内部的每个 I/O 上面加上一个 Boundary Scan cell 用于控制和观测每个 I/O 的状态, 然后把每个 I/O 的 bscell 串连起来交由 TAP 控制器控制。TAP 控制器按照 IEEE1149.1 规范通过 5 个 I/O 串行与外界通讯。虽然 Boundary Scan 是为了解决板级芯片互联的测试而设计的, 但是也可以用来进行芯片自身 I/O 的测试, 比如 VIH/VIL, VOH/VOL, highz, I/O 漏电等等的测试, IEEE1149.1 规范比较实用于纯数字电路的 I/O 测试, 后来为了解决一些高速差分接口的测试, 增加了 IEEE1149.6 规范, 通过在原有协议的基础上增加了两条指令 pulse 和 transition 来测试差分信号。关于 AC jtag 部分, 由于在芯片设计和板级设计上都有一些特殊考虑, 挑战会较大一些。

Boundary scan 是附加在芯片 I/O 周边的扫描测试链, 它通过专门的测试端口(TAP)访问。在测试模式下, 边界扫描链会接管功能逻辑, 对 I/O 进行灵活访问。边界扫描链的结构, 测试端口, 以及其控制器(TAP Controller), 被 IEEE 定为标准协议(IEEE 1149), 也称做 JTAG。边界扫描链最早应用于印刷电路板上芯片间的互联测试, 后来也广泛应用于生产测试中对芯片管脚的测试。

难度较大的部分:

越来越多的 MCP 产品测试需要, 在 die 的不同阶段 BScan 的不同测试方法和 pattern 准备;

除了基本的 In/Out/OEN 的测试外, 对 PAD 其他控制信号的测试实现。

2: MBIST 是什么？应用场景是什么？实现的方法是什么？挑战是什么？

答: MBIST 是 memory build-in self test, 就是对芯片中嵌入式的 memory 进行自测试。芯片中嵌入式的 memory 被深深埋到芯片的逻辑当中, 根本没有办法直接通过外部加激励来进行测试, 必须在芯片中设计一部分电路自动对嵌入式的 memory 进行自测试。在测试模式下, 它会接管功能逻辑对 memory 的控制, 依据特定算法, 对 memory 进行读写测试操作, 判断 Memory 是否有制造缺陷。现在 SoC 设计, 嵌入式的 memory 所占面积比例非常高, 通常一款 SoC 芯片, 75%~80% 的面积都是 embedded memory, 并且 embedded memory 的 density 很高, 在生产当中很容易产生 defect, 并且工艺不同, 产生 defect 的 density 容量也不同, 必须把所有有 defect 的芯片通过测试筛选出来, 这样 MBIST 几乎是每款芯片都要做的, 对于一些 memory 占比高的芯片, 不仅要 MBIST 还要做 Repair。

MBIST 的实现方法很多, 有很多成熟的 flow 可供采用, 根据芯片的特点定制适合的 flow。Flow 的定制也受限于各种因素, 比如 memory Vendor, tool vendor 的选取, 芯片对 timing, area, power, reliability 的要求, 后端 routing 的限制等。一个好的 Flow, 可节省人力、缩短 schedule, 节省芯片面积。粗糙一点的 Flow, 相比费时、费力, 会导致芯片面积偏大、测试时间增多, 芯片的 cost 增加。

挑战:

1. 对于 Memory Vendor 来说, 在新工艺下, memory 可能会有新的失效机制, 如何设计有效的 MBIST 算法是一项重要的研究课题。
2. 从 EDA 角度看, 目前业界工具对常规 memory(SRAM, ROM, Register File 等)的支持已相对成熟。芯片公司对 EDA 提供针对特殊类型的 memory (如 TCAM, DRAM) 的测试方案的需求正日渐迫切。
3. 从芯片实现的角度看, 设计者需要考虑如何使得 MBIST 逻辑对芯片 PPA 的影响最小。对于

使用带 Redundancy Memory 的 design 来说, 要考虑自修复(self-repair), 也会增加设计和验证的难度。

难度较大的地方: diagnosis, auto-repair 和 FA。

3: MBIST RTL flow 跟 netlist flow 各有什么优劣? 更推荐哪种 flow? 为什么?

答: 对于 design service 公司, 多用 netlist flow, 可以提供很高的自动化程度。综合工具可以对设计中用到的所有 Memory 有一个全局把控, 根据时钟域, Floorplan, power domain, memory power consumption 等因素合理地对 Memory 分组, 自动制定 Test plan。且由于 design service 一般都看不到 code, 而且和客户之间的 delivery 都已事前切割的很清楚, 和前面反复的也比较少, netlist flow 也较简单, 做起来较容易。

对于 design house 公司, 多用 RTL flow, 优点: 在设计早期, 可以在 RTL Level 提早把 mbist 做进去, 验证、综合都可以和设计一起做, 效果好一些。但 RTL flow 复杂性会大一些。主要体现在 rtl coding style, design database 的管理, sdk 的处理等上面。

具体用那种 flow, 要根据该次项目的实际情况和整个 DFT flow 的具体情况来分析, 以真实项目经验来说, 两种还都在使用中的。

4: MBIST 时, Memory partition 要考虑哪些因素? 推荐的做法是什么?

A: 主要考虑 memory type, clock domain, physical location, power consumption 等, 做 mbist 的时候最好带着 def 做。

5: LBIST 是什么? 应用场景是什么? 实现的方法是什么? 挑战是什么?

答: LBIST (Logic Built-In Self Test) 是针对逻辑电路的自测试。测试激励由片上 PRPG (Pseudo-random pattern generation) 来产生。输出响应通过 MISR (Multiple Input Signature Register) 来压缩, 最后对得到的特征值进行比对。LBIST 多应用于对可靠性要求较高的芯片 (如汽车电子, 工业级应用) 的系统自检测试。LBIST 产生的激励是随机的, 所以天生的缺陷是测试覆盖率不充分, 通过在设计中增加测试点 (Test Point Insertion) 可以在一定程度上得到改善。LBIST 不依赖于 ATE 机台, 直接在板上进行测试, 甚至芯片部署到产品中以后也可以进行 in-system 的测试。LBIST 的开发难度大、开发周期长, 对芯片面积也有很大的开销, 一般都是对可靠性要求比较高的芯片才做 LBIST, 像消费类的产品除非特殊要求, 一般不做 LBIST 的。

挑战: 由于 LBIST on chip 产生的激励是伪随机的, coverage 一般不高, 并且对芯片中的 X-source 敏感, 芯片中的 X-source 会直接 crash LBIST, 所以进行 lbist 设计, 清理芯片中的 x-source 是很大一部分工作量。

6: Analog 的 DFT 如何实现? 常用的结构有哪些? 挑战是什么?

答: Analog 的 DFT 测试一直都是一个难点, 现在 Analog 的测试都还是要依靠 designer 在电路是添加测试电路, 完全依赖于 designer 实现的。Analog DFT, 常常也把一些数模混合的 IP 归集到这一范畴, 这部分基本不太可能依赖 EDA tool 来实现, 往往都基于 RTL coding 完成。Analog/IP DFT 比较常用的架构有 Internal/External Loopback, JTAG program test, IO pin test muxing, embedded reg/mem sampling 等等。这部分重点是需要和 Analog/IP designer 紧密合作, 共同确定后续 lab 和量产测试的 spec; 还有就是因为包含了 Analog 部分, 如何完成仿真验证需要特别注意。

7: 常说的 scan chain, stuck-at 跟 at-speed test 各是什么? 分别如何实现? 挑战是什么?

答: Scan chain 即是由芯片中的寄存器串在一起组成的扫描链; stuck-at 和 at-speed 指的是 ATPG 当中的 fault model。Scan chain insertion 同样是靠工具完成。stuck-at pattern 是用来测试电路中 stuck 类型的 defect, 比如 open, short, tie high, tielow 等。At-speed test 主要是用来测试电路中 timing 相关的 defect。ATPG pattern 的产生主要是依赖于 EDA 工具来实现。设计的挑战是需要 DFT 工程师对设计非常了解, 理解 SoC 的设计, 特别需要了解 Clock domain 的控制及 CDC path 的处理等, 这些都是 design dependent 的, 所以 DFT 工程师要对设计非常了解才行。

8: 压缩解压缩的概念是什么? 为什么引入压缩解压缩? 如何定义压缩比?

答: 芯片设计规模愈来愈大, 受到管脚数目的限制, 传统的扫描链变得越来越长, 导致测试时间无法承受。缩短扫描链, 减少扫描链的数目, 是减少测试时间的根本手段, 由此压缩解压缩技术应运而生。如果没有压缩技术, Scan chain 会非常长, 相应的在 ATE 上的 test time 就长, 而 ATE test time 是影响整个芯片成本的一个关键因素。

具体压缩比的定义, 还是要看是用那种架构来实现。比如 M 的 TK, 由于它使用 LFSM 电路结构, 所以它能够做到比较高的压缩如数百倍或更高, 工具同时支持用户利用 analyze 方式来提前评估各压缩比例的优劣。近年来 S 和 C 家也改进了压缩解压缩的电路结构, 加入了时序逻辑到压缩解压缩电路中, 压缩比都能做到足够高。但过高的压缩率反而会使得覆盖率降低, 测试时间变长, 得不偿失。压缩比的高低受到 compression 架构的限制, 目前业界卓越的 compression 架构可以做到几百甚至上千的压缩比。过高的压缩比会带来严重的绕线拥堵, 对物理实现带来挑战。

9: ATPG 是什么? 用于 DFT 的哪个方面? 在电路结构上如何体现? 如今业界算法是否大同小异? 最流行的算法是什么?

答: ATPG 是 Autotest Pattern Generation, 指工具应用相应算法产生测试向量的过程, 用于利用 scan chain 来测试数字电路中的 defect, 做 ATPG 先决条件必须是插好 scan chain。ATPG 算法研究的是, 针对电路中的故障, 如何激发故障、观测故障效应等。教科书级别的算法有 D 算法和 PODEM 等。各 EDA 厂商 ATPG 算法的差异性主要是由不同的 Compression 架构导致。不同的工具竞相追逐的主要目标是在达到相同测试覆盖率的前提下, 谁的测试时间最短, 从而为客户节省成本。ATPG 工具的 runtime 也是评价 ATPG 工具的重要指标之一。ATPG 算法现已相当成熟, 各家的 EDA 工具大同小异, 没有太大差别。

10: OCC/OPCG/scan clock mux 用途是什么? 工作原理是什么? 如何实现?

答: OCC (On-Chip Clock) 或 OPCG (On-Product Clock Gating) 是为了做 at-speed 测试, 在设计中增加的时钟控制模块。这些指的都是同一个东西, 不同的叫法而已。(前提: 芯片的运行速度愈来愈快 GHz, 但测试机台的发展却相对滞后 200MHz, 这就需要利用片上 PLL 提供 at-speed 测试时钟) 是为了 at-speed ATPG 测试时在 function clock 和 shift clock 之间切换的一个控制逻辑, 本质就是一个 2 选一的 clock mux。

它的基本原理是在 scan shift 模式下, 选通慢速的 ATE 时钟, load 或 unload 扫描链; 在 capture 模式下, 对 free-running PLL clock 过滤筛选出 launch 和 capture clock 进行 at-speed, 测试设计时需要注意处理一下 cdc 的 path, 不要产生 glitch 就行。

在实现方式上, 前端设计公司大多用 RTL hand coding module 在 clock gen 设计过程中就放入 RTL database 当中去; 如果用 EDA tools 实现的话, S 家的 OCC 跟 C 家的 OPCG 都是

比较普遍的做法。

11: 当前 DFT 的发展趋势是什么？

答: DFT 是一个实战性很强的工作, 它是为了解决芯片设计、制造的问题而存在的, 他不可能独立发展, 它要伴随着芯片的发展而发展。假如存在一个理想情况, foundry 足够厉害, 生产的芯片 100% 是 no defect 的, 在使用过程中也不会引入新的 defect, 那 DFT 就没有必要做了, 费人费力费面积。如果从发展需求方向看, 由于国内芯片行业发展的非常快, 已经开始涉及对芯片可靠性要求非常高的行业, 比如工业控制, 汽车电子等产品, 这些产品对芯片的可靠性要求较高, 一般情况是需要对芯片做 in-system 的测试, 这就要求 DFT 越来越向 system 靠拢, 需要 DFT 工程师对 system 要熟悉。

挑战:

1. 对于 Mix-signal 的设计, 受到芯片管脚较少的限制, 如何做 low-Pin-Count Test 是一个挑战;
2. 对于汽车电子而言, 出于安全考虑, 设计者考虑更多的则是如何将测试覆盖率做到极致;
3. 消费电子的规模愈来愈大, 受到芯片管脚数目, ATPG runtime 以及测试功耗等因素的约束, 传统的 Flat-ATPG 已不再适用。层次化测试 (Hierarchical Test) 成为主流。层次化测试不仅可以允许不同模块灵活分组测试, 而且在 block level 产生的测试向量可以在顶层复用, 减少 ATPG 的 runtime;
4. 一些新技术的出现, 如 3D Stack Die, 如何做 TSV test, 也给 DFT 带来新的课题。

12: 16/12、7nm 对于 dft 有什么挑战, 或者有什么需要注意的地方吗, 从方法学和 flow 的角度?

答: 从两个方面来说: 其一, 工艺节点本身发展带来的挑战。主要是指工艺节点的演进引入的新的 defect model 需要新的 DFT 算法来检测。老实说对于这一部分大部分工程师或者公司无能为力, 因为需要 foundry 支持, 只有大公司才有可能获得 foundry 的合作。所以只能依赖于 EDA 工具来 support。其二是由于工艺节点的演进带来的集成度的提高, 使得 SoC 规模越来越大, SoC 集成的东西也越来越多, 同时 SoC 面临的 time-to-market 的压力也最大, 对于这一部分, 工程师能够支配的空间就很大, 一个完整、完备、自动化的 flow 对于提高交付的质量是一个很好的保证。

13: 请问在插入扫描链后, 再次编译的意义是什么呢? 是为了查看是否有 setup 违例和 holdup 违例吗? 编译前是否需要 sourceDFT 的时序约束呢? 还是使用 function 模式的时序约束?

答: 新加入了一些逻辑, 编译是为了将待测设计放入 TB 中, 从而查看是否有 setup 违例和 holdup 违例, 需要 sourceDFT 的时序约束及 function 模式的时序约束。

14: 什么是测试时间? 测试时间跟哪些因素相关? 缩短测试时间的办法有哪些?

答: ATE 测试程序会分别针对芯片 IO, 逻辑电路, Memory 以及模拟电路进行全面的测试。测试数据量的大小, 测试向量运行频率, 如何最大限度的并行测试, 都是影响测试时间的重要因素。同时, 在测试机台上尽可能做 Multi-site 测试, 即多块芯片同时测试, 也是缩短平均测试时间, 降低成本的重要手段。

测试时间相关因素有: SoC 的规模, Test Plan 和 DFT Spec 的要求, DFT design 实现架构, 具体 DFT 各项实现方法, DFT mode SDC 和 timing signoff, 还有 Physical floor plan 和 IR drop

影响等等。

减少测试时间和成本的关键就是首先 case by case 定义好该芯片的 DFT Spec, 然后基于芯片实际情况选择最合理的 design flow 和 EDA tools, 与 STA 工程师一起评估和定义高效而又合理的 SDC, 结合后端的情况和将来 ATE pattern program 的考虑, 调整 DFT design 情况和后面的 pattern 生成等等。

15: MBIST 时, Memory partition 要考虑哪些因素? 推荐的做法是什么?

答:

- 1.memory instance 在 floor plan 中的 location (DEF + more) , 主要是解决绕线长度(timing / transition) 和 congestion 问题;
- 2.memory 所处的 power domain 和 clock domain, 主要是满足 at speed test 要求;
- 3.memory type 和 Group balance, 主要是优化 test time, 同时也考虑 physical routing 和测试 power consumption/IR drop 等等;
- 4.其他还需要参考的因素还有部分 User-defined/programming Algorithm 要求, MBIST diagnosis 以及 memory redundancy repair 要求等等

16: Scan clock 的 leading-edge 和 trailing-edge 的定义是?

答: scan clock 的 leading-edge 和 trailing-edge 要区别于 rising edge 和 falling edge。

主要取决于你如何定义 scan clock, 他的 off state 是什么: leading-edge 是指 scan clock 从 off state 到 on state 的转换; falling-edge 是指 scan clock 从 on state 到 off state 的转换;

举例:

如果 clock 的定义的 off state 是 0, 那么 leading-edge 就是 0->1 的沿, 即可认为 rising edge, trailing-edge 为 1->0, 即可认为是 falling edge

17: mbist 算法有哪些? 都有啥作用?

答: 主要分为 Classical Algorithms 和 March Algorithms。

checkerboard 算法: 基本原理是对存储单元进行赋值, 保证每个单元与相邻单元格的值不同, 进行两次读写, 并验证。

March C 算法: 按固定的读写顺序进行操作。

March C: (w0); 向上(r0,w1); 向上(r1,w0); (r0) 向下; (r0,w1); 向下(r1,w0); (r0);

March C-: (w0); 向上(r0,w1); 向上(r1,w0); (r1w0); 向下(r0,w1); 向下(r1,w0); (r0);

March C+: (w0); 向上(r0,w1,r1); 向上(r1,w0,r0); (r0); 向下(r0,w1,r1); 向下(r1,w0,r0); (r0)

March C-改进了 C, 删掉了步骤 4 中的读 0, 但并没有减小覆盖率; C+在 C-的基础上进行了改进, 在每次写操作后增加了读操作, 增加了覆盖率。

18: ATPG 的思想是什么? 有什么算法?

答: 自动测试 pattern 生成技术, 测试向量按顺序地加载到器件的输入脚上, 输出的信号被收集并与预算好的测试向量相比较从而判断测试的结果。

算法: D 算法 (主流)、PODEM 算法、FAN 算法。

19: ATE 机台的测试流程?

答: 1. wafer 出场后需要进行 wafer 基本的测试, 标记出坏片。

2. 将测试通过的芯片切割并封装就可以得到一粒一粒单独的芯片。

3. 对封装后的芯片进行再一次的测试, 筛选掉封装后损坏的芯片。

4. 将测试通过的芯片交付客户。

20:LFSR 是什么？如何理解

答:LFSR (linear feedback shift registers),线性反馈移位寄存器。在 LBIST 中通常使用这类结构作为 TPG 生成用于生成穷举测试、伪随机测试和伪穷举测试的 test pattern 或 test sequence。

21:DPPM 是什么？

答:表示每百万片里有缺陷芯片的数量。

22:常见的物理缺陷有哪些？

答:由于电路缺陷而导致的，如电路的开路、短路、桥接等缺陷。大致可划分为原材料不良、异物、划伤、Bump 元件缺陷（凸起、错位或缺失）、金属性污染物和蚀刻液脏污残留、工程师操作失误。此外，芯片制造过程中还会产生如蚀刻锈斑、电镀过多、异色和金属线损坏等小样本缺陷

23:流片发现问题怎么办？

答:查找同批次和有嫌疑的芯片截留，同时分析原因，对工艺进行排查，找到受影响的批次隔离，评审对已出货的是否召回

24:有几种方法修 hold？

答:1.增大 Tdp;2.增大 Tlaunch;3.减小 Tcapture;4.插 lockup latch;5.局部 Useful skew

25. how do you evaluate whether the synthesis result is good or not?

答: PTA 满足设计需求（电耗、时序、面积）

26. could you let me know the purpose of post simulation?

答: 模拟真实环境，相对于前仿来说加入了线上的延时，从而模拟真实环境。

27. 为什么需要打断 loop？

答: loop 指的就是电路中存在的回环，指的是一个模块中输出对其自身的输入产生不可忽略的影响，并会对下级电路产生固定的值，使部分电路不可测，我们需要加入一些组合逻辑，打断回环，达到电路可测的目的。

28. 解释一下 generate_clock, 什么时候需要 define 一个 generate_clock?

答: 建立时钟，debug 时需要。

29. 为什么在前端设计电路时主要关注 setup time 而不是 hold time?

答: 因为 hold 可以由后端工程师修改，而要修改 set up time 只能由前端改逻辑或者加寄存器或者在后端降低频率（一般不会降低频率）。

30. 解释一下 setup time 和 hold time?

答: Tsu:在触发器的捕获沿到来之前，数据必须提前保持稳定的持续时间，以保证触发器可以顺利安全的取到数据;

Th:在触发器的捕获沿到来之后数据必须保持稳定的持续时间。

31. 假设有三级时序逻辑，中间有组合逻辑，如果中间这级有 hold 违例，你会发现一个什

么状况？

答：提前采了后一个值，中间少采上一个值。

32. 怎么修复 set up 违例？

答：前端改设计或加逻辑门，后端做降频处理。

33. 拿到一个 design，想把里面所有 memory 都抓出来应该用什么命令？

答：get_memory_instances

34. 为什么需要使用 formality 这个工具？

答：在插入测试逻辑后，与原设计进行对比，检查是否有更改原先设计。

35. DFT 网表与 fun RTL 做 formal 比较需要加哪些约束？

答：ijtag_rst(jtag 网络的复位信号)

36. EDT 电路的作用？

答：相对于传统的电路来说，具有缩短扫描链，减少扫描链的数目，减少测试时间的优点。

37. 为什么需要 at-speed 测试？

答：测试延迟故障，利用内部 PLL 高速时钟，进行快速测试。

38. MBIST 经验介绍一下？MBIST 在做什么？

答：MBIST 是 memory build-in self test，就是对芯片中嵌入式的 memory 进行自测试。芯片中嵌入式的 memory 被深深埋到芯片的逻辑当中，根本没有办法直接通过外部加激励来进行测试，必须在芯片中设计一部分电路自动对嵌入式的 memory 进行自测试。在测试模式下，它会接管功能逻辑对 memory 的控制，依据特定算法，对 memory 进行读写测试操作，判断 Memory 是否有制造缺陷。

39. MARCH C 和 checkerboard 算法在干嘛？

答：checkerboard 算法：基本原理是对存储单元进行赋值，保证每个单元与相邻单元格的值不同，进行两次读写，并验证；MARCH C：按固定的读写顺序进行操作，(w0);向上(r0,w1);向上(r1,w0);(r0)向下;(r0,w1);向下(r1,w0);(r0)。

40. formal 比较可以查出哪些问题？

答：设计功能发生改变，是否在可接受范围内。

41. 了解 bsdI 文件吗？干嘛用的？

答：边界扫描描述语言，是对边界扫描器件的边界扫描特性的描述，主要用来沟通边界扫描器件厂商、用户与测试工具之间的联系。

42. 边扫九大指令分别有什么作用？为什么需要？

答：

1) Extest 指令--强制指令,用于芯片外部测试，如互连测试

- 2) Sample/Preload 指令--强制指令,在进入测试模式前对 BSC 进行预装载
- 3) Bypass 指令--强制指令,提供穿透芯片的最短通路。
- 4) INTEST 指令 ---可选指令,选择 INTEST 指令时, IC 工作在边界扫描内部测试模式 (internal boundary-test mode), 选择边界扫描寄存器连通 TDI 和 TDO。
- 5) RUNBIST 指令 ---可选指令, 选择 RUNBIST 指令时, IC 工作在自测试模式 (self-test mode), 对 IC 的内部逻辑进行全面的自测试, 通过选择用户自定义的数据寄存器连通 TDI 和 TDO。
- 6) IDCODE 指令 ---可选指令, 选择 IDCODE 指令时, IC 工作在正常工作模式, 选择数据识别寄存器 (data identification register) 连通 TDI 和 TDO。
- 7) USERCODE 指令 ---可选指令, 选择 USERCODE 指令时, IC 工作在正常工作模式, 选择用户自定义数据寄存器 (user defined test data register) 连通 TDI 和 TDO。
- 8) CLAMP 指令 ---可选指令, CLAMP 指令使 IC 的输出置于由边界扫描寄存器的当前内容决定的逻辑电平上, 选择 BYPASS 寄存器连通 TDI 和 TDO。
- 9) HIGHZ 指令 ---可选指令, HIGHZ 指令使 IC 的所有输出置于高阻状态, 选择 BYPASS 寄存器连通 TDI 和 TDO。

43.IDDQ 原理?

答: 检测 CMOS 电路静态时的漏电流, 电路正常时静态电流非常小, 但在有缺陷时, 漏电流会增大数倍, 可用 IDDQ 的方法检测出来, 可检测出桥接、短路、栅氧短路等物理缺陷。

44.check_timing 是在检查什么?

答: 时序, setup timing、hold timing

45.常用的 Bscan Cell 有哪些?

答: 一般 input 和 output 使用 bc_1, inout 使用 bc_7, inout 的 oe 使用 bc_2

BSCAN Cell 全部类型

BC_1, BC_2, BC_2_A, BC_2_A_EXT, BC_2_B, BC_3, BC_4, BC_5,

BC_7, BC_7_LOW, BC_8, BC_9, BC_10:

46.了解 CP/FT 测试吗? 展开说说?

答: CP:晶圆测试; FT: 封装后测试。针对封装好的 chip, CP 测试之后会进行封装, 封装之后进行 FT 测试, CP 测试, 仅仅用于基本的连接测试和低速的数字电路测试, FT 测试在最短的时间内保证出厂的 Unit 能够完成全部的功能。

47.stuck-at、transation、path-delay 三种故障类型解释?

答: 在 Memory Fault Model 故障模型中 stuck-at 指存储单元中的值固定为 0, 称为 SA0; 存储单元中的值固定为 1, 称为 SA1。; transation Fault 是指一个存储单元中的数据无法从 0 转换到 1 或无法从 1 转换到 0。存储器中的转换故障有两类, 上转换故障和下转换故障。上转换故障时值无法从 0 跳变到 1, 下转换故障时值无法从 1 跳变到 0; 在 Logic Fault Model 中, Single stuck-at fault (SSF) 指逻辑门中一条信号线的逻辑状态, 一直固定在 0 或 1, 与其他信号线上的逻辑值无关; Transition delay fault (TDF) 是因逻辑门故障节点延时, 导致未能在期望时间内观测到正确的输出结果; Path delay fault (PDF): 指定路径上所有组合门电路的跳变延时之和的故障, 导致未能在期望时间内观测到正确的输出结果。

48.scan 的时候比如 clock gating 的控制是怎么做的？跟设计怎样合作的？

答：用 ICG，ICG 具有 TE 及 E 端，DFT 用 TE 端，设计用 E 端，TE 与 E 为或的关系。

49.介绍一下 jtag 1149 协议以及 jtag 1687 协议？

答：IEEE1149:标准测试访问端口和边界扫描架构，定义了可以包含在集成电路中的测试逻辑，以提供标准化的方法；IEEE1687:嵌入在半导体器件中的仪器的访问和控制标准，通过 IEEE 1149.1(TM)测试访问端口(TAP)和可能需要的额外信号，开发了一种访问嵌入式仪器的方法，该方法的元素包括特性的特征描述语言，以及与特性通信的特征描述语言，以及与特性交互的需求。

50.scan 的 at-speed 测试有哪些过程？

答：慢钟 shift, capture 阶段，第一个快钟 launch 数据产生一个翻转，第二个时钟 capture, 再对比数据。

51.锁存器和触发器的区别？

答：

锁存器：直接由激励信号控制电路状态的存储单元；

触发器：除激励信号外，还包含一个时钟的控制信号输入端，由时钟与激励信号一起控制电路状态。

52.MBIST 诊断流程是怎么实现的？

答：流片后或小规模量产，总是出现特定区域或者特定类型的 memory fault,判断故障原因（设计、机台、DFT）、确定错误原因。

53.memory 有哪些常见的故障？

答：Stuck-at fault、Transition fault、Coupling fault、Address-decoder fault。

54.为什么要选择 SMarchCHKBvcd 这种算法？单双口 sram 测试有什么区别？

答：SMarchCHKBvcd 算法覆盖率较高，可测的故障类型多；单口 SRAM 需要复用，只能在读操作后再进行写操作，双口可边读边写，提高测试速度。

54.综合完成之后的 check_timing 报告里你主要关注哪些问题？

答：no-clock, unconstrained, loops

55.ptpx 流程大概讲一下？

答：在 PT 上引入一个波形文件（验证），进行功耗分析

56.7T 和 9T 这两种工艺区别？

答：7T 指 7 个 track, stand cell（可以理解为布线通道）的高度。power, performance 和 area 结果不一样。

57.AC DC 覆盖率分别是多少？

答：AC:85% DC:99% 车载芯片要求会更高。

58.介绍一下什么是综合？

答：将功能验证没有问题的代码映射为数字逻辑电路。

59.pt px 中 saif 文件为什么需要？

答：saif 文件记录翻转率

60.说一下 memory 的耦合故障？

答：Coupling Fault 包含：CFst (state coupling fault)状态耦合故障、CFin(inversion coupling fault)倒置耦合故障、CFid(idempotent coupling fault)固化耦合故障

61.MBIST 控制器周围的电路如何保证覆盖率？

答：插链，做 DC 测试

62.PLL 怎么控制的？有哪些测试方式？

答：通过分频和倍频来得到我们想要的频率，在配置后观测输出频率。

63.IDDQ pattern 需要考虑电路结构吗？

答：不需要，我们只需要测 CMOS 的漏电流，不需要考虑电路结构。

64.memory 测试就用一种算法吗？没有补充算法吗？

答：看设计需要，一般情况下调用算法库，不需要补充。

